Swinburne University of Technology

Faculty of Science, Engineering and Technology

ASSIGNMENT COVER SHEET

Subject Code: COS30008
Subject Title: Data Structures and Patterns
Assignment number and title: 3, List ADT
Due date: March 22, 2026, 23:59
Lecturer: Dr. Bui Van Vuong

Your name: Nguyen Duc Manh **Your student id:** 105547489 / SWH02701

Check Tutorial	Mon 10:30	Mon 14:30	Tues 08:30	Tues 10:30	Tues 12:30	Tues 14:30	Tues 16:30	Wed 08:30	Wed 10:30	Wed 12:30	Wed 14:30

Marker's comments:

Problem	Marks	Obtained
1	48	
2	28	
3	26	
4	30	
5	42	
Total	174	

Extension certification:

This assignment has been given an extension and is now due on _____

Signature of Convener:_____

```cpp

// COS30008, List, Problem Set 3, 2022

#pragma once

#include "DoublyLinkedList.h"
#include "DoublyLinkedListIterator.h"

#include <stdexcept>

template<typename T>
class List
{
private:
    // auxiliary definition to simplify node usage
    using Node = DoublyLinkedList<T>;

    Node* fRoot;    // the first element in the list
    size_t fCount;  // number of elements in the list

public:
    // auxiliary definition to simplify iterator usage
    using Iterator = DoublyLinkedListIterator<T>;

    ~List() // destructor - frees all nodes
    {
        while (fRoot != nullptr)
        {
            if (fRoot != &fRoot->getPrevious()) // more than one element
            {
                Node* lTemp = const_cast<Node*>(&fRoot->getPrevious()); // select last

                lTemp->isolate(); // remove from list
                delete lTemp; // free
            }
            else
            {
                delete fRoot; // free last

```

```
                    break;                                                  ↵
                    // stop loop
            }
        }
    }

    void remove(const T& aElement)                                  //      ↵
      remove first match from list
    {
        Node* lNode =                                                       ↵
          fRoot;                                            // start at     ↵
          first

        while (lNode != nullptr)                                      //    ↵
          Are there still nodes available?
        {
            if (**lNode == aElement)                                  //    ↵
              Have we found the node?
            {
                                                                            ↵
                    break;                                                  ↵
                    // stop the search
            }

            if (lNode != &fRoot->getPrevious())                       //    ↵
              not reached last
            {
                lNode = const_cast<Node*>(&lNode->                          ↵
                  getNext());           // go to next
            }
            else
            {
                lNode =                                                     ↵
                  nullptr;                                          // stop  ↵
                  search
            }
        }

        // At this point we have either reached the end or found the node.
        if (lNode != nullptr)                                         //    ↵
          We have found the node.
        {
            if (fCount != 1)                                          //    ↵
              not the last element
            {
                if (lNode == fRoot)
                {
                    fRoot = const_cast<Node*>(&fRoot->                      ↵
                      getNext());       // make next root
```

```
                }
            }
            else
            {
                fRoot = nullptr;                                    // list becomes empty
            }

            lNode->isolate();                                       // isolate node
            delete lNode;                                           // release node's memory

            fCount--;                                               // decrement count
        }
    }

    ////////////////////////////////////////////////////////////////
    //// PS3
    ////////////////////////////////////////////////////////////////

    // P1

    List() : fCount(0), fRoot(nullptr) {};            // default constructor

    bool empty() const { return fCount == 0; };       // Is list empty?
    size_t size() const { return fCount; };           // list size

    void push_front(const T& aElement) {
        Node* aNode = new Node(aElement);
        if (fRoot != nullptr) {
            fRoot->push_front(*aNode);
        }
        fRoot = aNode;   // the recently added one will now be the root
        fCount++;
    };

    Iterator begin() const
    {
        return Iterator(fRoot).begin();
    }

    Iterator end() const
    {
```

```
        return Iterator(fRoot).end();
    }

    Iterator rbegin() const
    {
        return Iterator(fRoot).rbegin();
    }

    Iterator rend() const
    {
        return Iterator(fRoot).rend();
    }

    // P2

    void push_back(const T& aElement) {
        Node* aNode = new Node(aElement);
        if (fRoot != nullptr) {
            fRoot->push_front(*aNode);
        }
        else {
            fRoot = aNode;  // empty list case, same as push_front
        }
        fCount++;
    };

    // P3

    const T& operator[](size_t aIndex) const {
        if (aIndex >= fCount) {
            throw std::out_of_range("Index out of bounds.");
        };

        const Node* currentNode = fRoot;
        for (size_t i = 0; i < aIndex; i++) {
            currentNode = &currentNode->getNext();
        };

        return **currentNode;
    };

    // P4

    List(const List& aOtherList)
        : fRoot(nullptr), fCount(0)
    {
        *this = aOtherList;
    };

```

```
    List& operator=(const List& aOtherList) {
        if (this == &aOtherList) {
            return *this;
        }

        this->~List();
        fRoot = nullptr;
        fCount = 0;
        // Copy each element from that list to the copy
        for (auto i = aOtherList.begin(); i != aOtherList.end(); ++i) {
            this->push_back(*i);
        }
        return *this;
    };

    // P5

    List(List&& aOtherList)
        : fRoot(aOtherList.fRoot), fCount(aOtherList.fCount)
    {
        aOtherList.fRoot = nullptr;
        aOtherList.fCount = 0;
    };

    List& operator=(List&& aOtherList) {
        if (this == &aOtherList) {
            return *this;
        }

        this->~List();
        fRoot = nullptr;
        fCount = 0;

        fRoot = aOtherList.fRoot;
        fCount = aOtherList.fCount;

        aOtherList.fRoot = nullptr;
        aOtherList.fCount = 0;

        return *this;
    };

    void push_front(T&& aElement) {
        Node* aNode = new Node(std::move(aElement));
        if (fRoot != nullptr) {
            fRoot->push_front(*aNode);
        }
        fRoot = aNode;   // the recently added one will now be the root
        fCount++;
```

```
    };

    void push_back(T&& aElement) {
        Node* aNode = new Node(std::move(aElement));
        if (fRoot != nullptr) {
            fRoot->push_front(*aNode);
        }
        else {
            fRoot = aNode;  // empty list case, same as push_front
        }
        fCount++;
    };
};


```